CODEROOM

Project Blueprint & Development Roadmap

A browser-based platform for temporary rooms where people join with a five-digit code and share real-time messages, notes, images, PDFs, and videos.

Product concept

Create a room, share its short code verbally or in person, and let friends join from different devices and networks. Participants can chat and share files without exchanging usernames, contact details, or invitation URLs. Rooms and their files expire automatically.

Chosen product requirements

Temporary rooms • Five-digit access codes • Anyone with the code can attempt to join • Server-mediated file uploads and downloads • Automatic room and file expiration

This document covers the product scope, technology stack, system architecture, database design, API plan, security model, retention strategy, hosting considerations, and phased development roadmap. It intentionally excludes source code.

1. Product Overview

CodeRoom is designed for quick, temporary sharing. For example, a student in a library can create a room, tell a friend the five-digit code, and share class notes, photos of a whiteboard, PDFs, or a lecture video. Multiple people can participate in the same room.

Core MVP features

- Create a temporary room and display a five-digit numeric code.
- Join a room using the code and a display name.
- Send and receive text messages in real time.
- Upload and download images, PDFs, documents, and videos.
- Show connected participants and basic join/leave events.
- Display upload progress, file names, and file sizes.
- Load recent message history after refresh or reconnect.
- Expire rooms automatically and remove associated files.
- Support desktop and mobile browsers with a responsive interface.

Features to postpone until later

- Voice and video calls, user accounts, contact lists, and social features.
- End-to-end encryption, advanced search, and complex moderation tools.
- Peer-to-peer file transfer or multi-region infrastructure.

Important access principle

A five-digit code is convenient, but it is not strong authentication. Treat it as a room locator and add rate limiting, room-scoped participant sessions, and abuse controls.

2. Recommended Technology Stack

Layer	Recommendation	Responsibility
Frontend	React + Vite + TypeScript	Responsive UI, room screens, chat, previews, upload progress, WebSocket client.
Backend API	Python + FastAPI	Room lifecycle, access checks, messaging, file upload/download, quotas, cleanup coordination.
Real-time channel	WebSockets	Live message delivery, participant presence, room events, reconnect behavior.
Database	PostgreSQL	Rooms, participants, messages, file metadata, status, expiry timestamps.
Object storage	Cloudflare R2	Stores actual file contents; database stores metadata and private object keys.
Development	Docker + GitHub	Repeatable local environment, version control, testing, and deployment workflow.

Why this stack?

It is affordable to prototype, works with common web-development skills, and keeps responsibilities separate. PostgreSQL stores structured records, while object storage holds large files. For the MVP, Redis, Kubernetes, microservices, and a dedicated message broker are not required. Add infrastructure only when measured traffic or reliability requirements justify it.

Useful documentation

- React documentation
- FastAPI documentation
- FastAPI WebSockets
- PostgreSQL documentation
- Cloudflare R2 documentation

3. System Architecture and Data Flow

The browser communicates with the FastAPI application over HTTPS for API operations and WSS for live messaging. The application uses PostgreSQL for structured data and R2 for file bytes. A scheduled cleanup worker expires rooms and removes their stored objects.

Users' browsers React interface on desktop and mobile
HTTPS / WSS Encrypted API requests and WebSocket connections
FastAPI application server Room access, message persistence and broadcast, upload/download handling, rate limits
PostgreSQL • Room and message metadata
Cloudflare R2 • Actual image, video, and document bytes
Expiration worker Revoke access, delete objects, and clean database records

Text-message flow

1. A participant sends a message through an authenticated WebSocket connection.
2. The server validates the participant session and confirms the room is still active.
3. The server stores the message in PostgreSQL.
4. After a successful database write, the server broadcasts the message to connected participants in that room.
5. Clients can request message history after a refresh or reconnect.

File-sharing flow

1. The browser uploads a file to an authenticated backend endpoint.
2. The backend validates the room, participant, size limit, quota, and file type.
3. The backend streams the file to private object storage without loading the entire file into memory.
4. The server saves file metadata in PostgreSQL and announces the attachment to room participants.
5. Recipients request downloads through an authorized endpoint. Later, short-lived signed storage URLs may reduce backend bandwidth, but that changes the file-transfer path.

Scaling note

A single application instance is adequate for an MVP. If the application later runs multiple server processes or instances, use shared coordination or a pub/sub layer for cross-instance WebSocket broadcasts.

4. Database Design

Use four core relational tables. Store file contents in object storage rather than PostgreSQL. Add foreign keys, suitable cascade rules, transactions, and indexes during implementation.

Table	Key fields	Purpose
rooms	id (UUID), code_hash, created_at, expires_at, last_activity_at, status	One row per room. Status can be active, expired, deleting, or deleted.
participants	id (UUID), room_id, display_name, token_hash, joined_at, last_seen_at	A participant session within a room; no account or email is required for the MVP.
messages	id (UUID), room_id, participant_id, content, created_at, deleted_at	Text-message history. Index room_id and created_at for efficient retrieval.
files	id (UUID), room_id, participant_id, object_key, original_name, mime_type, size_bytes, created_at, deleted_at	Attachment metadata. Keep storage object keys private and never trust client-chosen paths.

Relationships

- One room has many participants, messages, and files.
- One participant can send many messages and upload many files.
- Every message and file belongs to a specific room.
- A file ID alone must never grant permission to download the file.

Recommended indexes and constraints

- Unique index on code_hash to prevent duplicate active room-code records.
- Index on messages (room_id, created_at).
- Indexes on rooms (status, expires_at) and files (room_id).
- Foreign keys between child records and rooms/participants.
- Use database transactions for operations that create related records.

5. Room-Code Design and Initial Limits

Five numeric digits provide 100,000 possible codes, including codes with leading zeros. The short code is easy to communicate but can be guessed by automated attempts, so access controls are essential.

1. Generate the number with a cryptographically secure random generator and format it as exactly five digits.
2. Create a keyed hash of the code on the server and check for collisions before creating the room.
3. Apply rate limits to room creation and join attempts.
4. Issue a separate random participant session token after a successful join. Store its hash and scope it to the room.
5. Validate session tokens and room expiry on API requests and WebSocket connections. Avoid putting bearer tokens in URLs.

Setting	Suggested initial value
Default room lifetime	2 hours
Maximum room lifetime	8 hours

Setting	Suggested initial value
Maximum participants	20
Maximum file size	25 MB
Maximum total uploads per room	100 MB
Failed join attempts	5 per minute per IP as an initial limit
Maximum message length	4,000 characters

These values are starting points for testing, not universal security standards. Tune them based on user experience, hosting limits, and abuse patterns.

6. API and Real-Time Event Plan

Method	Endpoint	Purpose
POST	/api/rooms	Create a room and return its code and initial session information.
POST	/api/rooms/join	Join using the five-digit code and a display name.
GET	/api/rooms/{id}	Retrieve room details after authorization.
GET	/api/rooms/{id}/messages	Load recent message history.
POST	/api/rooms/{id}/files	Upload a file after authorization and validation.
GET	/api/files/{id}/download	Download a file after verifying room membership and expiry.
DELETE	/api/rooms/{id}/leave	Leave a room and update participant presence.
WS	/api/rooms/{id}/ws	Maintain the real-time connection for messages and room events.

Suggested WebSocket events

Event	Direction	Purpose
message.send	Client → server	Submit a text message.
message.new	Server → room	Broadcast a stored message.
file.ready	Server → room	Announce a successfully uploaded attachment.
participant.joined	Server → room	Notify members of a new participant.
participant.left	Server → room	Notify members when a participant leaves.
room.expired	Server → room	Tell connected clients to close the session.

Authorization rule

Every endpoint and WebSocket connection that accesses a room must verify the participant session, room membership, and room status. Never rely only on a room ID or file ID supplied by the browser.

7. Security and Privacy Model

Area	Controls
Room isolation	Scope every message, file, download, and WebSocket event to the authorized room. Test for cross-room access.
Upload safety	Allowlist types, inspect file signatures, enforce size limits, generate random storage keys, and consider malware scanning.
Transport and storage	Use HTTPS/WSS, keep object storage private, protect secrets, and use secure session handling.
Abuse prevention	Rate-limit code guessing, message frequency, room creation, and uploads. Cap concurrent connections and per-room quotas.

Area	Controls
Browser safety	Render messages as text, avoid injecting user HTML, escape display names and filenames, and set safe download headers.
Logging	Never log participant tokens, storage secrets, or unnecessary sensitive content.
Public service operations	Offer an abuse-reporting path and a way to remove harmful content if the service is open to the public.

Privacy limitations to communicate clearly

- A room code is not strong authentication; anyone who obtains it may be able to join.
- Transport encryption is not the same as end-to-end encryption. The server handles message and file contents in this architecture.
- Deleting a room should revoke access immediately, but physical deletion can take time if storage operations fail.
- Do not promise instant or forensic-grade erasure from backups, logs, or provider infrastructure. Publish a truthful retention policy.

8. Automatic Room and File Expiration

A database expiration timestamp does not delete objects from storage on its own. Use a scheduled cleanup worker, initially running every minute, and keep the process retryable.

Step	Action
1. Detect	Find rooms whose expires_at timestamp has passed.
2. Revoke	Mark each room as deleting and block new messages, uploads, downloads, and connections.
3. Delete objects	Remove all stored objects associated with the room from object storage.
4. Delete records	Remove messages, participant records, and file metadata, or apply the chosen retention policy.
5. Complete	Mark the room as deleted once cleanup is successful.
6. Reconcile	Periodically detect orphaned objects, stale records, and abandoned multipart uploads.

Reliability requirements

- Make deletion idempotent so retrying an operation is safe.
- If storage deletion fails, retain a deleting state and retry later.
- Enforce expiration during every access check, even if the cleanup worker has not run yet.
- Avoid extending room expiry indefinitely through trivial activity unless that is an explicit product choice.
- Define whether messages, logs, and backups follow the same retention schedule as files.

9. Hosting and Cost Strategy

Component	Possible starting option	Considerations
Frontend	Cloudflare Pages	Free static hosting may be enough for the UI.
Backend	Render or another application host	Free tiers may sleep, limit compute, or be unsuitable for production.
Database	Supabase or managed PostgreSQL	Review storage limits, connection limits, backup policies, and free-tier expiry.
File storage	Cloudflare R2	Review storage, request, and operation pricing; free allowances can change.
Cleanup worker	Scheduled task or cron-capable service	Must run reliably and support retries; verify the selected host's scheduling limits.

Official pricing and service references

- Cloudflare Pages
- Render free services and limitations
- Supabase pricing
- Cloudflare R2 pricing

Video traffic is likely to be the largest variable cost. For example, 100 people each downloading a 20 MB video ten times generate about 20 GB of downloads. The backend-mediated design means backend bandwidth and compute

limits matter too. Start with conservative quotas, monitor real traffic, and use spending alerts or limits where available.

Free-tier reality check

A free prototype is realistic, but free hosting is not a guarantee of unlimited availability, storage, bandwidth, or permanent database retention. Confirm current provider limits before launch.

10. Development Roadmap

Stage	Estimated time	Work	Deliverable
1. Interface	1–2 days	Landing page, create/join screens, responsive room layout, mock data.	Clickable frontend.
2. Rooms and database	2–3 days	PostgreSQL schema, migrations, room codes, joining, session tokens, expiry.	Two browsers join the same room.
3. Real-time chat	2–3 days	WebSockets, message persistence, broadcast, presence, reconnect and history.	Live chat that survives refresh.
4. File sharing	3–4 days	R2 integration, streaming upload/download, previews, progress, quotas.	Participants exchange files.
5. Security and cleanup	2–3 days	Rate limits, validation, access checks, expiration and retryable deletion.	Expired rooms become inaccessible and files are cleaned up.
6. Testing	2–3 days	Test disconnects, invalid codes, concurrent uploads, expired rooms, abuse cases.	Repeatable test checklist.
7. Deployment	1–2 days	Deploy components, configure HTTPS, secrets, monitoring and alerts.	Public MVP.

Time estimates are approximate and assume focused work. Allow extra time for learning, debugging, deployment issues, and provider configuration.

Recommended implementation order

1. Create a room and join it from a second browser.
2. Send messages in real time and recover history after refreshing.
3. Add file uploads and authorized downloads.
4. Add expiration enforcement and reliable object deletion.
5. Harden security, test failure cases, and deploy.

11. Pre-Launch Test Checklist

- Two different networks can join the same room.
- Messages arrive in real time and survive a page refresh.
- Files download only for authorized room participants.
- Invalid codes and excessive join attempts are rejected.
- Oversized and unsupported files are rejected.
- Expired rooms reject new connections and downloads.
- Failed file deletion is retried successfully.
- Room codes and participant tokens are not exposed in logs.
- The application works on desktop and mobile.
- Usage limits prevent a single room from exhausting storage.

Operational checks

- Monitor backend errors, upload failures, storage usage, and cleanup backlog.
- Test how the application behaves when the database or object storage is temporarily unavailable.
- Document incident response, data retention, and abuse-reporting procedures before a public launch.

12. Final Recommendation

Build CodeRoom as a modular monolith first: one FastAPI application, one PostgreSQL database, private object storage, and one scheduled cleanup process. This is simple enough for an MVP but has a clear path to growth.

The first milestone should be deliberately small: create a room, join from another browser, exchange messages, and recover history after refresh. Once that works reliably, add file storage and then automatic expiration. This keeps debugging focused and lets each major feature be tested independently.

Architecture decisions at a glance

React + Vite + TypeScript • FastAPI • WebSockets • PostgreSQL • Cloudflare R2 • Five-digit room codes • Room-scoped participant sessions • Server-mediated uploads and downloads • Scheduled expiration and deletion

Scope note

This blueprint describes the system and its development plan. It intentionally contains no implementation source code. Technology choices and provider pricing should be checked against current documentation before deployment.